

we use the dictionary for a standard deck of values. We build a deck of 52 cards, saved in `self.startingDeckList`, by iterating through all suits, then iterating through all card ranks and values. In the inner `for` loop ❸, we use a call to the `items()` method of a dictionary that allows us to easily get the key and value (here, the `rank` and `value`) in a single statement. Each time through the inner loop we instantiate a `Card` object, passing the rank, suit, and value of the new card. We append each `Card` object to the list `self.startingDeckList` to create a full deck of cards.

The final step is to call the `shuffle()` method ❹ to randomize the deck. The purpose of this method may seem obvious: to shuffle the deck. However, it does an extra little trick. The `__init__()` method built the `self.startingDeckList`, and that work should only be done once. So, whenever we shuffle the deck, rather than re-creating all the `Card` objects, we make a copy of the starting deck list, save it in `self.playingDeckList`, and shuffle that. The copy is what will be used and manipulated as the game runs. With this approach, we can remove cards from `self.playingDeckList` and not have to worry about adding them back into the deck later or reloading cards. The two lists, `self.startingDeckList` and `self.playingDeckList`, share references to the same 52 `Card` objects.

Note that when we call `shuffle()` for subsequent runs of the game, some of the `Card` objects may be in the “revealed” state. So, before proceeding, we iterate through the entire deck and call the `conceal()` method on each card, so that all cards will initially appear face down. The `shuffle()` method finishes by randomizing the cards in the playing deck using `random.shuffle()`.

The `getCard()` method ❺ retrieves a card from the deck. It first checks to see if the deck is empty and, if so, raises an exception. Otherwise, since the deck is already shuffled, it pops a card off the deck and returns that card to the caller.

Together, `Deck` and `Card` provide a highly reusable combination of classes that can be used in most card games. The Higher or Lower game only uses eight cards for each round and shuffles the entire deck at the start of each game. Therefore, in this game it is not possible for the `Deck` object to run out of cards. For a card game where you need to know if the deck runs out of cards, you can build a `try` block around the call to `getCard()` and use an `except` clause to catch an exception. The choice of what to do there is up to you.